

Zcash, end to end

from a wallet seed to a verified shielded payment

m@rek.onl

One order

Why nodes must agree on an order

Two transactions spend the same coin. Each is valid alone; together, one must lose.



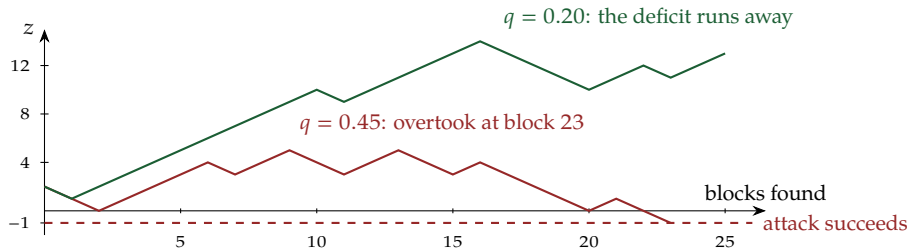
Consensus supplies a **canonical order**: the chain with the most work. Every later rule, the note tree, the nullifier set, the pool balances, is a function of that order.

The race

An attacker mines a secret branch; z is the honest chain's lead in blocks.

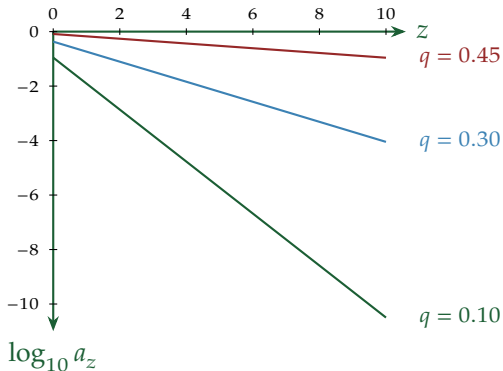
$$z_{i+1} = \begin{cases} z_i + 1 & \text{honest block, probability } p \\ z_i - 1 & \text{attacker block, probability } q = 1 - p \end{cases}$$

The attack succeeds when z reaches -1 .



The catch-up probability

$$a_z = \Pr[\text{the walk ever reaches } -1 \text{ from } z] = \begin{cases} (q/p)^{z+1} & q < p \\ 1 & q \geq p \end{cases}$$



q	a_0	a_5
0.10	0.111	1.9×10^{-6}
0.30	0.429	6.2×10^{-3}
0.45	0.818	0.30

Each block of lead multiplies a_z by q/p : the order of old blocks becomes final exponentially fast.

What the order gives the shielded protocol

object	a function of the ordered chain
note commitment tree anchor	every cmx, appended in order the tree's root after a block
nullifier set	every nf published so far
pool balance	minus the sum of every v_{balance}

From here on: one canonical chain. The rest of the talk follows one payment through it.

Keys

From the seed to an account key

$$I := \text{BLAKE2b-512}(\text{ZcashIP320rchain}, S), \quad (\text{sk}_m, c_m) := (I[0..32], I[32..64])$$

$$(\text{sk}_i, c_i) := \text{PRFexpand}_{c_{\text{par}}}([0x81] \parallel \text{sk}_{\text{par}} \parallel \text{I2LEOSP}_{32}(i)) \quad \text{split in halves, } i \geq 2^{31}$$

Path $m/32'/133'/a'$ (purpose, coin type, account):

node	spending key sk
m	7eee3c10...8e8b2cd4b8
$m/32'$	93af0a87...2dc9eb0f87
$m/32'/133'$	149fcee5...1a0bcb0e8c4c
$m/32'/133'/0'$	b67d8d87...8313cb223b4

S the seed bytes 00, ..., 1f; every step is one-way and hardened.

Spending secrets

$$\text{ask} := \text{ToScalar}(\text{PRFexpand}_{\text{sk}}([06])) \in \mathbb{F}_{p_{\text{Vesta}}}$$

$$\text{nk} := \text{ToBase}(\text{PRFexpand}_{\text{sk}}([07])) \in \mathbb{F}_{p_{\text{Pallas}}}$$

$$\text{r}_{\text{ivk}} := \text{ToScalar}(\text{PRFexpand}_{\text{sk}}([08])) \in \mathbb{F}_{p_{\text{Vesta}}}$$

$$\text{ak}^{\mathbb{P}} := [\text{ask}] G^{\text{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \quad (\text{even } y), \quad \text{ak} := \text{Extract}_{\mathbb{P}}(\text{ak}^{\mathbb{P}})$$

ask signs spends; nk keys nullifiers; r_{ivk} blinds the viewing key.

PRFexpand_k is BLAKE2b-512 personalised Zcash_ExpandSeed; the lead byte names the output.

Viewing keys

$$\text{ivk} := \text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{ak}, \text{nk}) = \text{Extract}_{\mathbb{P}}(\text{SinsemillaHashToPoint}(\text{ak} \parallel \text{nk}) + [r_{\text{ivk}}] H)$$

$$\text{dk} \parallel \text{ovk} := \text{PRFexpand}_{r_{\text{ivk}}}([82] \parallel \text{ak} \parallel \text{nk})$$

key	lets its holder
ask	spend
(ak, nk, r_{ivk})	see spends (nullifiers)
(dk, ivk)	see incoming notes, make addresses
ovk	recover what it sent

The address

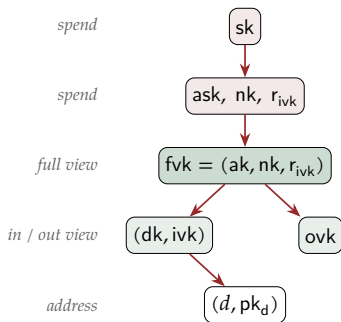
For an index j :

$$d := \text{FF1}_{\text{dk}}(\text{LE}_{88}(j)),$$

$$g_d := \text{GroupHash}(\text{z.cash:Orchard-gd}, d),$$

$$\text{pk}_d := [\text{ivk}] g_d$$

The address is (d, pk_d) .



- 2^{88} addresses per key, all with $\text{pk}_d = [\text{ivk}] g_d$.
- Two addresses of one key are unlinkable without dk or ivk .
- The address is all a sender needs.

Someone pays you

The note

The sender creates

$$n = ((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$$

with v the amount in zatoshi and $\rho := \text{nf}^{\text{old}}$, the nullifier the same Action spends.
From one fresh seed rseed :

$$\text{esk} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([04] \parallel \rho))$$

$$\psi := \text{ToBase}(\text{PRFexpand}_{\text{rseed}}([09] \parallel \rho))$$

$$\text{rcm} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([0B] \parallel \text{g}_d \parallel \text{pk}_d \parallel v \parallel \rho \parallel \psi))$$

One seed fixes the note's randomness and the encryption key; the recipient can re-derive all three.

The note commitment

$$\begin{aligned} \text{cm} &:= \text{NoteCommit}_{\text{rcm}}(g_d, pk_d, v, \rho, \psi) \\ &= \text{SinsemillaHashToPoint}(g_d \parallel pk_d \parallel v \parallel \rho \parallel \psi) + [\text{rcm}] H_{\text{cm}} \\ \text{cmx} &:= \text{Extract}_{\mathbb{P}}(\text{cm}) \end{aligned}$$

- **Hiding:** for uniform rcm , cm is a uniform point, whatever the note.
- **Binding:** two notes with one cmx give a discrete-log relation among the generators.
- cmx is appended to the note commitment tree; the note itself never appears.

Encryption to the address

$$\text{epk} := [\text{esk}] g_d, \quad s := [\text{esk}] \text{pk}_d,$$

$$K := \text{BLAKE2b-256}(\text{Zcash_OrchardKDF}, s \parallel \text{epk})$$

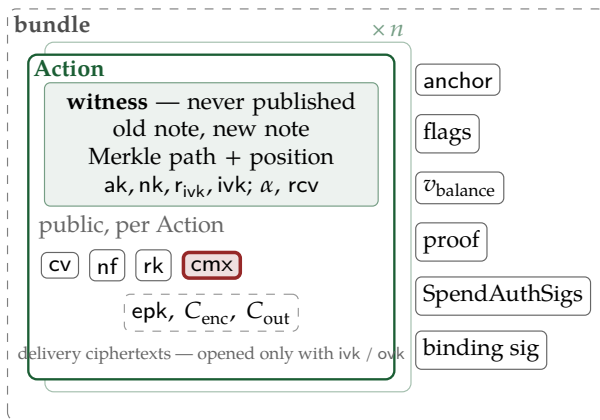
$$C_{\text{enc}} := \text{ChaCha20-Poly1305}_K(0x03 \parallel d \parallel v \parallel \text{rseed} \parallel \text{memo}) \quad (580 \text{ bytes})$$

The recipient holds ivk with $\text{pk}_d = [\text{ivk}] g_d$, so

$$[\text{ivk}] \text{epk} = [\text{ivk} \cdot \text{esk}] g_d = [\text{esk}] \text{pk}_d = s.$$

Diffie–Hellman on a per-address base: without esk , only ivk recovers K .

What reaches the chain



Per Action: cv^{net} , nf , rk , cmx , epk , C_{enc} , C_{out} (820 bytes). Nothing names the recipient.

Scanning

Compact blocks

A light wallet does not download full transactions. Per Action it receives

$$(\text{nf}, \text{cmx}, \text{epk}, C_{\text{enc}}[0:52]) \quad 32 + 32 + 32 + 52 = 148 \text{ bytes}$$

The first 52 bytes of the plaintext are $\text{leadByte} \parallel d \parallel v \parallel \text{rseed}$: everything but the memo.

The memo, the tag and C_{out} are fetched only for the wallet's own transactions.

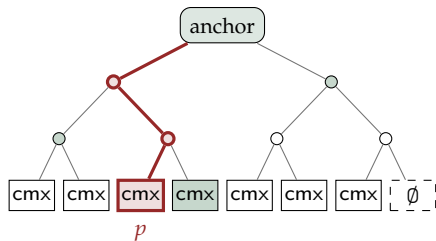
Trial decryption

For every Action, with the wallet's ivk:

1. $s := [\text{ivk}] \text{epk}$, $K := \text{KDF}(s, \text{epk})$, decrypt 52 bytes;
2. $g_d := \text{GroupHash}(d)$, $\text{pk}_d := [\text{ivk}] g_d$;
3. re-derive esk , ψ , rcm from rseed and $\rho = \text{nf}$ of the same Action;
4. accept only if $[\text{esk}] g_d = \text{epk}$ and $\text{Extract}_{\mathbb{P}}(\text{NoteCommit}_{\text{rcm}}(\dots)) = \text{cmx}$.

An Action for someone else decrypts to garbage and fails step 4. An accepted note is the unique opening of cmx : the sender cannot make the wallet accept a note it cannot later spend.

Keeping a witness



The note's cmx sits at position pos of a depth-32 tree. With $h_t := \text{MerkleCRH}(31 - t, \cdot, \cdot)$:

$$a_0 := \text{cmx}, \quad a_{t+1} := \begin{cases} h_t(a_t, s_t) & b_t = 0 \\ h_t(s_t, a_t) & b_t = 1 \end{cases}$$

Valid path: $a_{32} = rt$, the anchor.

Every sibling s_t is a function of the cmx values appended so far: the wallet updates the path from the blocks it scans and never reveals pos.

Watching for spends

The wallet computes its note's nullifier once:

$$\text{nf} := \text{Extract}_{\mathbb{P}}([(\text{PoseidonHash}(\text{nk}, \rho) + \psi) \bmod p_{\text{Pallas}}] K^{\text{Orchard}} + \text{cm})$$

and marks the note spent when nf appears in a scanned block.

- Deterministic: one note, one nullifier.
- Unlinkable: without nk it looks random.
- The wallet now holds everything a spend needs: the note, pos, the path, an anchor.

Spending

The spend side of an Action

$$\begin{array}{ll} \text{nf}^{\text{old}} := \text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm}) & \text{published} \\ \text{rsk} := \text{ask} + \alpha, \quad \text{rk} := \text{ak}^{\mathbb{P}} + [\alpha] G^{\text{SpendAuth}} & \text{fresh } \alpha \text{ per Action} \\ \text{cv}^{\text{net}} := [v^{\text{old}} - v^{\text{new}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}} & \text{fresh rcv} \end{array}$$

- rk is a uniform point: two spends by one key are unlinkable.
- cv^{net} is a uniform point: the amount is hidden, and commitments add up.

The output side

The new note goes to the payee, change to an internal address of the payer; each with

$$\rho^{\text{new}} := \text{nf}^{\text{old}}$$

so every created note carries a unique ρ . Unused sides are dummy notes of value 0: every Action has exactly one spend and one output.

Action	v^{old}	v^{new}	$v^{\text{old}} - v^{\text{new}}$
note $\rightarrow$ payee	7 ZEC	5 ZEC	2 ZEC
dummy $\rightarrow$ change	0	1.9999 ZEC	-1.9999 ZEC
v_{balance} (the fee)			0.0001 ZEC

What the proof asserts

For public $(rt, cv^{\text{net}}, nf^{\text{old}}, rk, cmx, \text{flags})$, the prover knows a witness such that:

1. the old note opens cm^{old} , and cm^{old} has a valid path to rt (if $v^{\text{old}} \neq 0$);
2. $nf^{\text{old}} = \text{DeriveNullifier}_{nk}(\rho, \psi, cm^{\text{old}})$;
3. $rk = ak^{\mathbb{P}} + [\alpha] G^{\text{SpendAuth}}$, and $pk_d^{\text{old}} = [\text{Commit}_{r_{ivk}}^{\text{ivk}}(ak, nk)] g_d^{\text{old}}$;
4. $cv^{\text{net}} = [v^{\text{old}} - v^{\text{new}}] V^{\text{Orchard}} + [rcv] R^{\text{Orchard}}$;
5. the new note opens cmx , with $\rho^{\text{new}} = nf^{\text{old}}$;
6. the flags hold (spends, outputs, cross-address).

One Halo 2 proof for all Actions of the bundle.

Signatures

Every signature signs SigHash, a BLAKE2b digest of the transaction without its proofs and signatures.

- **Spend authorisation**, per Action: RedPallas under rsk, checked against rk. Only a holder of ask can produce it.
- **Binding**, per bundle:

$$\text{bsk} := \sum_i \text{rcv}_i, \quad \text{bvk} := \sum_i \text{cv}_i - [v_{\text{balance}}] V^{\text{Orchard}}$$

RedPallas under bsk, checked against bvk.

Verification

What a node checks

check	guarantees
the proof, on the public fields	statement 1–6 for every Action
rt is a past anchor	the spent note exists
$nf \notin$ nullifier set	not spent before
spend signature under rk	the owner authorised it
binding signature under bvk	value is conserved

Then: every nf joins the set, every cmx joins the tree. No step reads a note.

Why the binding signature enforces balance

Commitments add:

$$\sum_i cv_i = \left[\sum_i (v_i^{\text{old}} - v_i^{\text{new}}) \right] V^{\text{Orchard}} + \left[\sum_i rcv_i \right] R^{\text{Orchard}}$$

so

$$bvk - [bsk] R^{\text{Orchard}} = \left[\sum_i (v_i^{\text{old}} - v_i^{\text{new}}) - v_{\text{balance}} \right] V^{\text{Orchard}}.$$

A valid signature proves knowledge of b with $bvk = [b] R^{\text{Orchard}}$. If the values did not balance, b would yield $\log_{R^{\text{Orchard}}} V^{\text{Orchard}}$, a relation between two hashed generators: infeasible.

The sum is below $2^{80} \ll p_{\text{Vesta}}$, so balance modulo p_{Vesta} is balance over the integers.

How Halo 2 proves

The claim becomes a table

"I know x with $x^3 + x + 5 = 35$ " over $\mathbb{F}_{97}$; the witness $x = 3$:

$$3 \xrightarrow{\text{square}} 9 \xrightarrow{\times 3} 27 \xrightarrow{+3} 30 \xrightarrow{+5} 35.$$

One operation per row, one gate equation on every row:

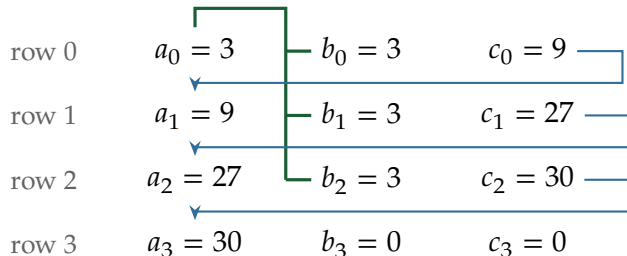
$$q_M a b + q_L a + q_R b + q_O c + q_C + \text{PI} = 0$$

every cell in $\mathbb{F}_{97}$.

row	advice			fixed					instance	
	a	b	c	q_M	q_L	q_R	q_O	q_C	PI	
0	3	3	9	1	0	0	-1	0	0	$x \cdot x = x^2$
1	9	3	27	1	0	0	-1	0	0	$x^2 \cdot x = x^3$
2	27	3	30	0	1	1	-1	0	0	$x^3 + x = s$
3	30	0	0	0	1	0	0	5	-35	$s + 5 = 35$

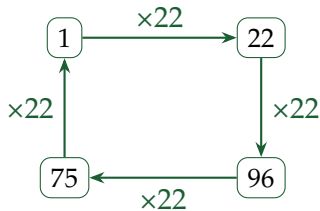
Advice: the prover's secret trace. Fixed: the circuit. Instance: the public 35. Copy constraints wire the rows; range checks are lookups into a table.

Equal values must be wired together



Equal numbers in a table are not a constraint. These *copy constraints* are enforced separately, by a permutation argument.

Give the rows multiplicative labels



$$\omega = 22, \quad \omega^2 = 96 = -1, \quad \omega^4 = 1$$

$$H = \{1, 22, 96, 75\} \subset \mathbb{F}_{97}$$

Row i gets the label ω^i : a cycle of field values.

Rows become polynomials

Row i lives at $\omega^i \in H$. Interpolate each column:

$$a(1) = 3, a(22) = 9, a(96) = 27, a(75) = 30 \implies a(X) = 90 + 61X + 22X^2 + 24X^3$$

$$G(X) := q_M ab + q_L a + q_R b + q_O c + q_C + \text{PI}$$

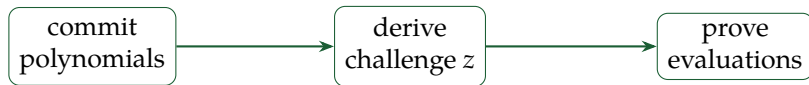
$a, b, c, q_\bullet, \text{PI} \in \mathbb{F}_{97}[X]$ of degree < 4 ; $G \in \mathbb{F}_{97}[X]$ of degree ≤ 9 .

$$Z_H(X) := \prod_{h \in H} (X - h) = X^4 - 1$$

$$\text{every gate holds} \iff G \text{ vanishes on } H \iff Z_H \mid G \iff G = t \cdot Z_H$$

A correct computation *is* a polynomial identity. A wrong one cannot be: the lie $x^2 = 10$ in row 0 leaves a remainder $24(1 + X + X^2 + X^3) \neq 0$.

One random point



1. The prover *commits* to a, b, c and t .
2. The challenge $z := H(\text{transcript})$ — Fiat–Shamir.
3. The prover opens $a(z), b(z), c(z), t(z)$.
4. The verifier checks one equation:

$$G(z) \stackrel{?}{=} t(z) Z_H(z)$$

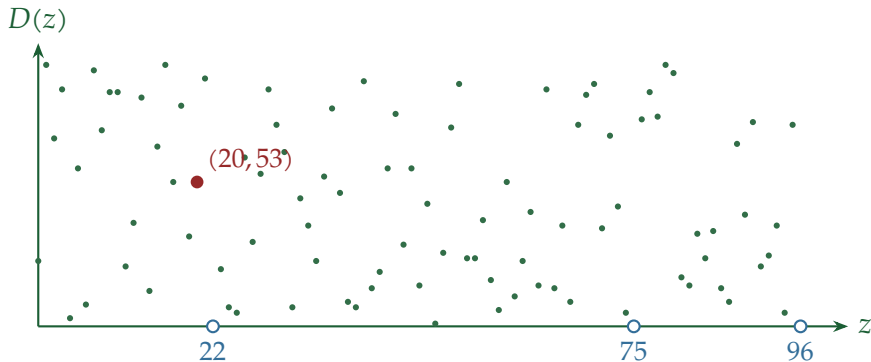
$z \in \mathbb{F}_{97}$ in the toy; $z \in \mathbb{F}_p$ in the protocol.

Honest, $z = 20$: $G(20) = 75 = 67 \cdot 46$. The row-0 lie: $G^*(20) = 20 \neq 64$.

A false identity is frozen before z is drawn; it survives only if z is one of its few roots.

A nonzero polynomial has few roots

The remainder $D(X) = 24(1 + X + X^2 + X^3)$ over $\mathbb{F}_{97}$: zeros at 22, 75, 96 only.



For uniform z this cheat passes with probability $3/97$; over the protocol's field, about 2^{-240} .

Committing to a polynomial

$$P := \langle \vec{a}, \vec{G} \rangle + [r] H, \quad \vec{a} = (a_0, \dots, a_{d-1}), \quad \vec{G} \text{ independent points}$$

$$a(z) = \langle \vec{a}, \vec{b} \rangle, \quad \vec{b} := (1, z, z^2, \dots, z^{d-1})$$

$a_i, r, z \in \mathbb{F}_p$; $\vec{G}, H, P \in \mathcal{U}$, the Vesta curve, whose scalar field is $\mathbb{F}_p$.

- A Pedersen commitment to the coefficient vector: hiding, binding.
- Opening at z is an inner product with a public vector.
- The problem: convince the verifier of $\langle \vec{a}, \vec{b} \rangle = v$ in fewer than d group elements, without sending $\vec{a}$.

The inner-product argument: one fold

Split every vector in half and publish two cross terms:

$$\begin{aligned} L &:= \langle \vec{a}_{\text{lo}}, \vec{G}_{\text{hi}} \rangle + [\ell] H + [\langle \vec{a}_{\text{lo}}, \vec{b}_{\text{hi}} \rangle] U, \\ R &:= \langle \vec{a}_{\text{hi}}, \vec{G}_{\text{lo}} \rangle + [r_R] H + [\langle \vec{a}_{\text{hi}}, \vec{b}_{\text{lo}} \rangle] U. \end{aligned}$$

Challenge u ; both sides fold:

$$\vec{a}' := u \vec{a}_{\text{lo}} + u^{-1} \vec{a}_{\text{hi}}, \quad \vec{G}' := u^{-1} \vec{G}_{\text{lo}} + u \vec{G}_{\text{hi}}, \quad \vec{b}' := u^{-1} \vec{b}_{\text{lo}} + u \vec{b}_{\text{hi}}$$

$$P + [v] U + [u^2] L + [u^{-2}] R = \langle \vec{a}', \vec{G}' \rangle + [r + u^2 \ell + u^{-2} r_R] H + [\langle \vec{a}', \vec{b}' \rangle] U$$

$$u \in \mathbb{F}_p^\times; L, R, U, H \in \mathcal{U}; \ell, r_R \in \mathbb{F}_p.$$

The same claim at half the length. The verifier folds $\vec{G}$ and $\vec{b}$ itself and never sees $\vec{a}$.

The recursion

After $k = \log_2 d$ rounds every vector has length 1. The proof: $2k$ points (L_j, R_j) , one scalar a , one blinder r^* .

$$P + [v] U + \sum_{j=1}^k ([u_j^2] L_j + [u_j^{-2}] R_j) \stackrel{?}{=} [a] G^{(0)} + [r^*] H + [a \cdot b^{(0)}] U$$

$a, b^{(0)}, r^*, u_j \in \mathbb{F}_p; G^{(0)}, L_j, R_j \in \mathcal{U}$.

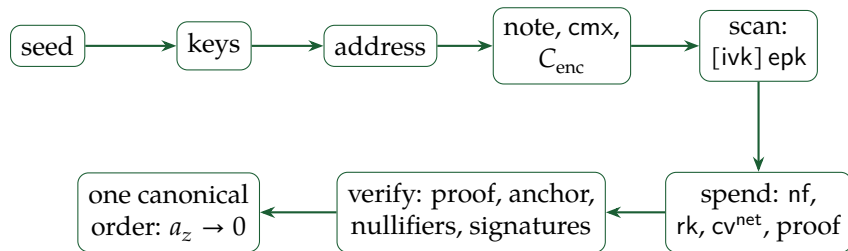
- The scalar $b^{(0)} = \prod_j (u_j^{-1} + u_j z^{2^{j-1}})$, with u_1 the last challenge: logarithmic work.
- The point $G^{(0)} = \langle \vec{s}, \vec{G} \rangle$: one multi-scalar multiplication, shared across a batch of proofs.
- Binding: two openings would give a discrete-log relation among $\vec{G}, H$; an extractor rewinds three challenges per round.

The Orchard circuit

- Size: 2^{11} rows, 10 advice columns; arithmetic over $\mathbb{F}_p$ so Pallas points cm, rk, cv are native; commitments on Vesta.
- Gadgets: Sinsemilla (commitments, tree), Poseidon (nullifier), elliptic-curve arithmetic, lookups for range checks.
- The instance: $anchor, cv, nf, rk, cmx, flags$.
- Generators from a hash: no trusted setup, no ceremony.
- Proof size $2720 + 2272a$ bytes for a Actions: 4992 for one.

Every clause of the statement is a set of rows; one proof, one random point, one inner-product argument.

The whole path



The verifier learns that every check passes, the fee, the anchor and the flags, and nothing about the notes.